

# Exam 1 foundation check answer guide

Use this after attempting the questions. A wrong answer is a starting point for review. A correct guess also deserves a second look.

## Check your reasoning

Compare your answer and explanation. Mark any question you missed, guessed, or cannot explain. Do not use a total score to decide whether you are ready for the exam.

### 1 Yes

Both names refer to one list. Appending changes that shared object. A “No” answer suggests reviewing references and mutation.

### 2 No

The work adds:  $n + n = 2n$ , which grows linearly. Consecutive loops do not multiply their iteration counts.

### 3 No

Python lists support direct indexing. Reading the last item takes  $O(1)$  time regardless of how many items come before it.

### 4 Yes

In place describes where sorting happens and how much extra space it uses. Returning the original list is a separate requirement of the in-place sorting exercises here.

### 5 A

The output is `[1, 2]`. The addition creates a new list and the assignment changes only the local name items. B confuses rebinding with mutation; C confuses the caller's list with the new item; D assumes valid code raises an error.

### 6 C

The inner work repeats  $n$  times for each of  $n$  outer iterations:  $n$  times  $n$ . B counts only one loop; A ignores repetition; D would need repeated halving.

### 7 B

A full array must copy its  $n$  existing items into larger storage before adding the new item, so one append can take  $O(n)$ . A ignores resizing; C adds an unnecessary second factor of  $n$ ; D confuses resizing with halving.

## Remaining answers

### 8 D

Size counts stored items; capacity counts available slots. A reverses the terms; B ignores unused slots; C counts unused slots as stored items.

### 9 B

Bo comes first because 1 is smaller than 2. Ada stays before Cy because equal keys keep their original order. A reverses that tie; C is not sorted; D treats stability as optional.

### 10 C

The pass goes  $[4, 1, 3, 2] \rightarrow [1, 4, 3, 2] \rightarrow [1, 3, 4, 2] \rightarrow [1, 3, 2, 4]$ . It places the largest item at the end, but need not finish the whole sort. A assumes one pass completes sorting; B skips the swaps; D reverses the comparison direction.

### 11 Output is 1 0

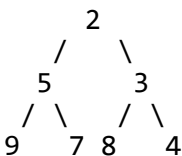
Each instance receives its own value attribute. Calling `a.add()` changes `a.value` only. An output of 1 1 suggests confusing separate instances with shared state.

### 12 Output is 3

The values after division are 4, 2, 1. Count the three divisions, not the four numbers including the starting 8. Repeated halving gives logarithmic growth.

### H1 Draw and update a heap

The initial tree is:



Text equivalent: root 2 has children 5 and 3; 5 has children 9 and 7; 3 has children 8 and 4.

Return 2. Move the last item, 4, to the root:  $[4, 5, 3, 9, 7, 8]$ . Swap with the smaller child, 3:  $[3, 5, 4, 9, 7, 8]$ . Stop because 4 is less than its only child, 8. Keeping the last slot empty preserves the complete-tree layout.

## Use the result

Start with the earliest area below that has a marked question. One mistake is a reason to investigate, not proof that the whole topic is weak.

- Questions 1, 5, 11: Topic 1 Python. Then try mock Q2 and Q3.
- Questions 2, 6, 12: Topic 2 complexity. Then try mock Q1 and Q3.
- Questions 3, 7, 8: Topic 3 arrays and lists. Then try mock Q1 and Q4.
- Questions 4, 9, 10, 13: Topic 4 sorting. Use question 13's explanations on the next page to identify the algorithm or reasoning step to review.
- Question H1: Topic 5 heaps, if included. Review array-to-tree mapping, then removal and sifting.

## 13 Sorting costs and reasoning

These are tight worst-case bounds for the stated implementations with distinct keys. A derivation must connect the algorithm's steps to the amount of work.

| Algorithm | Worst case    | Derivation or repeated work                                |
|-----------|---------------|------------------------------------------------------------|
| Selection | $O(n^2)$      | Scan lengths $n - 1, n - 2, \dots, 1$ .                    |
| Bubble    | $O(n^2)$      | Reversed input requires quadratically many adjacent swaps. |
| Insertion | $O(n^2)$      | Reversed input requires $1 + 2 + \dots + (n - 1)$ shifts.  |
| Merge     | $O(n \log n)$ | About $\log n$ levels, with $O(n)$ total work per level.   |
| Quick     | $O(n^2)$      | Extreme pivots give sizes $n, n - 1, \dots, 1$ .           |

### Example pseudocode and explanation

Selection sort is one valid choice:

```
FOR each position from first to next-to-last
  REMEMBER that position as the smallest so far
  SCAN every later position
    IF its value is smaller, remember its position
  SWAP the smallest value into the current position
```

Even on reversed input, it scans  $(n - 1) + \dots + 1$  positions:  $n(n - 1)/2$  comparisons. That sum grows quadratically. Any input order reaches this comparison count.

Other choices are valid if the pseudocode correctly describes the algorithm and the reasoning matches the table. Bubble and insertion reach quadratic work on reversed distinct input. Merge performs all splitting and merging levels. Last-pivot quick sort reaches quadratic work on sorted distinct input because each partition removes only one item from the next recursive call.

### Find the missing idea

- Wrong cost: trace a worst-case input and count the repeated comparisons or moves.
- Merge or quick errors: draw two recursion levels and explain how their sizes change.
- Correct cells but unclear reasons: connect each loop or recursive call in the pseudocode to the count. Recognizing a formula is not yet deriving it.

## Retry after reviewing

Close the review. Try the question for your area and explain your answer before checking below. Use a different example if you remember the answer without understanding it.

### Python

```
items = [4]
other = items
other.append(7)
other = [9]
print(items)
```

Output and reason: \_\_\_\_\_

### Complexity

Start question 12 with  $n = 16$ . How many divisions occur? What changes if  $n$  doubles again?

Answer and reason: \_\_\_\_\_

### Arrays and lists

An array has size 4 and capacity 4. It doubles when full. After appending one item, what are its size and capacity, and how many old items were copied?

Answer and reason: \_\_\_\_\_

### Sorting

Write the list after one left-to-right bubble-sort pass on [3, 1, 2]. Was a swap made? Can the early-exit flag stop after this pass?

Answer and reason: \_\_\_\_\_

### Optional heaps

Start with min-heap [2, 5, 4]. Append 1 and sift up. Write the resulting list.

Answer and reason: \_\_\_\_\_

## Check after trying

- Python: [4, 7]. Appending changes the shared list; rebinding other does not.
- Complexity: 4 divisions; doubling to 32 adds one division.
- Arrays: size 5, capacity 8, with 4 old items copied.
- Sorting: [1, 2, 3]. Swaps occurred, so the flag does not stop yet.
- Heaps: [1, 2, 4, 5]. The new item swaps with 5, then with 2.

If you can explain the retry, move to the matching coding task. If not, inspect one relevant hint or solution section, close it, and retry. Ask your TA with the exact step that confused you.